



Universidad Nacional del Sur

Ingeniería
Electrónica

Movilidad de mano impresa en 3D en base al sensado de los movimientos (Telerobótica)

Autores:
Joaquín Petruf
Sebastian Amoroso

Tutores:
Rodrigo Martin Santos
Gabriel Eggly

Proyecto Final

Bahía Blanca, Buenos Aires
October 12, 2026

Contents

1 Motivación **3**

2 Solución **3**

3 Desarrollo **4**
3.1 Transmisor 4
3.1.1 Diseño 4
3.1.2 Conexionado y Código 4
3.1.3 MPU6050 con Filtro de Kalman 7
3.2 Receptor 12
3.2.1 Diseño 12
3.2.2 Conexionado y Código 14

4 Pruebas **19**

5 Conclusiones **20**

1 Motivación

Vivimos en una era donde la tecnología esta presente en casi todos los ámbitos de nuestra vida, siendo de utilidad tanto para actividades cotidianas como para tareas que presenten dificultades dada la posibilidad de atentar contra nuestra salud o por limitaciones físicas, entre otras. Gracias a los avances que se han logrado y se siguen logrando, es posible dar soluciones que permitan y faciliten la realización de dichas tareas, sin importar el obstáculo que presente, se obtienen sistemas compactos y sencillos de utilizar.

Un ejemplo claro y directo de esta necesidad es la manipulación de materiales peligrosos. En escenarios que involucran sustancias tóxicas, reactivos químicos volátiles, material radiactivo o entornos con temperaturas extremas, la presencia humana es un riesgo inaceptable. Es una de muchas situaciones en la que es requerida la ayuda de la tecnología, y en especifico el sector al que apunta este proyecto, el de la Telerobótica, replicando los movimientos de una mano humana en una impresa en 3D. Nuestra motivación principal es demostrar cómo la réplica exacta del movimiento humano en una máquina a distancia puede ofrecer soluciones vitales en situaciones que presentan un alto riesgo o dificultad para las personas.

2 Solución

La Telerobótica es el área de la robótica concerniente al control de robots a distancia, mediante conexiones inalámbricas, como WIFI, Bluetooth, etc. La idea es que un operador humano maneje dispositivo mecánico o autónomo situado en un lugar diferente, a veces peligroso o de difícil acceso. Se usan consolas de control que replican los movimientos de forma precisa y devuelven información visual o táctil.

La base de este proyecto es censar los movimientos de una mano humana y poder replicarlo, de forma inalámbrica, en una mano impresa en 3D. Los tipos de movimientos que se replicaran son:

- Apertura y cierre de los dedos, de forma individual.
- Rotación de muñeca.
- Desplazamiento vertical de la mano.

Para esto se hará uso del microcontrolador ESP NodeMCU-32S, 5 servos SG90 y 2 servos MG946R, potenciómetros y el modulo MPU6050.

El micro ESP32 presenta la flexibilidad de poder realizar una comunicación inalámbrica, mediante WiFi, logrando hasta una distancia de 200 metros en espacio libre. Se utilizaran los servos SG90 para poder realizar los movimientos de los dedos, como en este proyector no se contempla la acción de fuerza de la mano se utilizan estos por ser de bajo torque y dimensiones pequeñas, luego, para la rotación y desplazamiento vertical de la mano si se utiliza los MG946R que son de alto torque, ya que para estas acciones si se necesitan mas fuerza por parte del servo. Cabe destacar que todo esto es por parte del receptor, el cual se deberá alimentar con una fuente que pueda dar la potencia necesaria para que la micro y servos puedan trabajar con normalidad.

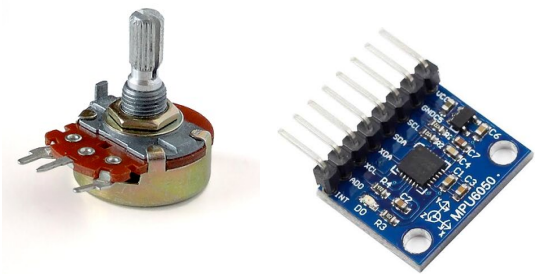


(a) ESP-32S (b) SG90 (c) MG946R

Figure 1

Por parte del transmisor, se hará uso de un sistema que mediante el movimiento vertical del dedo hará rotar un potenciómetro, el cual censará la posición, la micro lee este valor y lo transmite por Wifi. Ademas del censado de los dedos, se utiliza el acelerómetro y giroscopio MPU6050, con el cual se podrá medir el ángulo de

inclinación y rotación de la mano, se debe tener en cuenta que este debe estar posiciona de forma paralela a la mano y bien orientado, ya que un mal posicionamiento influirá en su medición.



(a) Potenciometro (b) MPU6050

Figure 2

3 Desarrollo

Los detalles del desarrollo del sistema se dividirá en dos partes: El receptor y el transmisor.

3.1 Transmisor

3.1.1 Diseño

3.1.2 Conexionado y Código

Pasando al desarrollo del código

```
#include <esp_now.h>
#include <WiFi.h>
#include "MPU6050_Filtro.h"

// MAC de la placa RECEPTORA
uint8_t macReceptor[] = {0x1C, 0xC3, 0xAB, 0xD2, 0x14, 0x9C};
```

se utilizan distintas librerías para el funcionamiento del sistema, entre estas "ESP-NOW" y "WiFi" serán las encargadas de la comunicación inalámbrica entre las micro, su importancia se explicara mas adelante en la sección del Receptor. Luego, se importa una librería local creada para el manejo del giroscopio y acelerómetro MPU6050, se detallara su funcionamiento mas adelante en la presente sección.

Para la transmisión es muy importante que la micro conozca la MAC de la ESP que va a recibir los datos, de esta manera se asegura que los datos seran recibidos de forma eficiente en el destinatario deseado.

```
const int pinPulgar = 39;
const int pinIndice = 34;
const int pinMedio  = 35;
const int pinAnular = 32;
const int pinMenique= 33;

// --- VARIABLES PARA EL SUAVIZADO DE POTENCI METROS (Filtro EMA) ---
const float ALFA_ADC = 0.15;
float filtroPulgar, filtroIndice, filtroMedio, filtroAnular, filtroMenique;

typedef struct mensaje_brazo {
  uint16_t valPulgar;
  uint16_t valIndice;
  uint16_t valMedio;
```

```
15   uint16_t valAnular;
16   uint16_t valMenique;
17   uint16_t valMuniecaVertical;
18   uint16_t valMuniecaRotacional;
19 } mensaje_brazo;
20
21 mensaje_brazo datosAEnviar;
22 esp_now_peer_info_t infoReceptor;
```

Se definen los pines utilizados para la lectura de los valores de los potenciómetros, se debe tener en cuenta que todos estos pines pertenezcan al ADC1 (Conversor analógico-digital), ya que la micro presenta dos conversores, pero el segundo se bloquea cuando se hace uso del WiFi.

Se crean variables para el filtrado de la señal leída de los potenciómetros, para evitar enviar paquetes basuras si el pote tiene falso contacto.

Para los paquetes a enviar se crea la estructura "mensaje-brazo", el cual debe ser igual tanto en el receptor como el transmisor, instanciandolo después. Ademas, en la línea 22 se crea el "perfil" del destinatario, ya que, antes de enviar el paquete se debe registrar la configuración del "peer".

```
1  void setup() {
2    Serial.begin(115200);
3
4    // 1. Configuracion explicita del ADC antes de encender la radio
5    pinMode(pinPulgar, INPUT);
6    pinMode(pinIndice, INPUT);
7    pinMode(pinMedio, INPUT);
8    pinMode(pinAnular, INPUT);
9    pinMode(pinMenique, INPUT);
10   analogReadResolution(12);
11   delay(100);
12
13   // 2. Inicializar Comunicaciones
14   WiFi.mode(WIFI_STA);
15   WiFi.setSleep(false); // Desactiva el ahorro de energia
16
17   if (esp_now_init() != ESP_OK) {
18     Serial.println("Error inicializando ESP-NOW");
19     return;
20   }
21   esp_now_register_send_cb((esp_now_send_cb_t)OnDataSent);
22
23   memcpy(infoReceptor.peer_addr, macReceptor, 6);
24   infoReceptor.channel = 0;
25   infoReceptor.encrypt = false;
26   esp_now_add_peer(&infoReceptor);
27
28   // 3. Inicializar el MPU6050
29   inicializarMPU();
30
31   // 4. Valores iniciales para el filtro EMA
32   filtroPulgar = analogRead(pinPulgar);
33   filtroIndice = analogRead(pinIndice);
34   filtroMedio  = analogRead(pinMedio);
35   filtroAnular = analogRead(pinAnular);
36   filtroMenique = analogRead(pinMenique);
37
38   // ========================================
```

```
  // WARM-UP (PRECALENTAMIENTO DE FILTROS)
  // ========================================
  Serial.println("Estabilizando filtros matematicos...");
  for (int i = 0; i < 50; i++) {
    filtroPulgar = (analogRead(pinPulgar) * ALFA_ADC) +
                   (filtroPulgar * (1.0 - ALFA_ADC));

    filtroIndice = (analogRead(pinIndice) * ALFA_ADC) +
                   (filtroIndice * (1.0 - ALFA_ADC));

    filtroMedio  = (analogRead(pinMedio) * ALFA_ADC) +
                   (filtroMedio  * (1.0 - ALFA_ADC));

    filtroAnular = (analogRead(pinAnular) * ALFA_ADC) +
                   (filtroAnular * (1.0 - ALFA_ADC));

    filtroMenique = (analogRead(pinMenique) * ALFA_ADC) +
                   (filtroMenique * (1.0 - ALFA_ADC));

    leerAngulosFiltrados();
    delay(20);
  }
  Serial.println("Transmisor Listo.");
}
```

Ya definiendo el hardware de la micro en el setup() lo primero que se hace es definir los pines en modo entrada, y asignarle la resolución al conversor, que en esta caso sera de 12 bits.

En la siguientes 10 líneas se hace el encendido del WiFi, poniendo la antena en modo estación, en caso de no hacer esto la placa podría actuar como un router. Ademas, se apaga el ahorro de energía para garantizar que el envío de datos sea continuo y sin latencia.

Luego, se copia la MAC de la placa receptora, se le asigna cualquier canal de WiFi disponible, y se desactiva la encriptación para poder ganar velocidad para el envió de los datos. Continuamos inicializando el modulo MPU6050, el cual se explicará a detalle mas adelante.

Se continua tomando los primeros valores en crudo de los potenciómetros, mediante un ciclo for, se realizan varias iteraciones aplicando el filtro EMA, de esta manera se busca estabilizar los valores leídos por la ESP32 antes de enviar los paquetes al receptor, pudiendo asegurar que los valores enviados no son lecturas falsas.

Por ultimo, se vera como funciona el loop(), el cual se estará ejecutando constantemente.

```
  // 1. Filtrado en tiempo real de los analogicos
  filtroPulgar = (analogRead(pinPulgar) * ALFA_ADC) +
                 (filtroPulgar * (1.0 - ALFA_ADC));

  filtroIndice = (analogRead(pinIndice) * ALFA_ADC) +
                 (filtroIndice * (1.0 - ALFA_ADC));

  filtroMedio  = (analogRead(pinMedio) * ALFA_ADC) +
                 (filtroMedio  * (1.0 - ALFA_ADC));

  filtroAnular = (analogRead(pinAnular) * ALFA_ADC) +
                 (filtroAnular * (1.0 - ALFA_ADC));

  filtroMenique = (analogRead(pinMenique) * ALFA_ADC) +
                  (filtroMenique * (1.0 - ALFA_ADC));

  // 2. Empaquetar datos de los dedos
  datosAEnviar.valPulgar  = (uint16_t)filtroPulgar;
```

```
19    datosAEnviar.valIndice  = (uint16_t)filtroIndice;
20    datosAEnviar.valMedio   = (uint16_t)filtroMedio;
21    datosAEnviar.valAnular  = (uint16_t)filtroAnular;
22    datosAEnviar.valMenique = (uint16_t)filtroMenique;
23
24    // 3. Empaquetar datos del MPU6050
25    AngulosMano angulos = leerAngulosFiltrados();
26    datosAEnviar.valMuniecaVertical = angulos.vertical;
27    datosAEnviar.valMuniecaRotacional = angulos.rotacion;
28
29    Serial.print("Pulgar:");
30    Serial.print(datosAEnviar.valPulgar);
31    Serial.print(", Indice:");
32    Serial.print(datosAEnviar.valIndice);
33    Serial.print(", Medio:");
34    Serial.print(datosAEnviar.valMedio);
35    Serial.print(", Anular:");
36    Serial.print(datosAEnviar.valAnular);
37    Serial.print(", Menique:");
38    Serial.print(datosAEnviar.valMenique);
39    Serial.print(", Munieca_Vertical:");
40    Serial.print(datosAEnviar.valMuniecaVertical);
41    Serial.print(", Munieca_Rotacional:");
42    Serial.println(datosAEnviar.valMuniecaRotacional);
43
44    // 4. Enviar
45    esp_now_send(macReceptor, (uint8_t *) &datosAEnviar, sizeof(datosAEnviar));
46
47    delay(20); // ~50Hz de actualizacion
48  }
```

En cada ciclo lo primero que se realiza es la lectura de los potenciómetros, para aplicar instantáneamente el filtro EMA, el cual es utilizado para reducir el ruido como se mencionara varias veces, de esta manera se estabilizan los valores. Una vez realizado el calculo se empaqueta lo que se obtuvo para poder enviárselo al receptor, la forma de asignar los valores es mediante "Casteo" colocando el tipo de variable "uint16_t" por delante del nombre de cada una.

También se deben tomar los valores para la rotación y desplazamiento vertical de la mano, estos se lo piden a la librería creada para el manejo del modulo MPU6050, los cálculos se realizan ahí mismo, por lo tanto, solo hay que hacer la asignación directa para poder enviarlos.

El uso de "Serial.print" es para poder determinar de forma visual por el monitor serie si los valores que se enviaron son los que se esperan. Una vez realizado todo lo anterior, mediante la función "esp_now_send" se le da como argumento la estructura de datos a enviar para que los convierta en un tren de datos en bytes.

Se debe destacar el uso del "delay(20)", este ayudara al manejo de transmisión en la red, ya que si no se usase el ESP32 intentaría enviar los paquetes a la velocidad del procesador, lo que saturaría el buffer de radio. Ademas, se sincroniza dichos pulsos de trabajo con el de los servomotes que trabajan a 50 Hz (20ms), buscando eliminar latencias.

3.1.3 MPU6050 con Filtro de Kalman

A continuación se explicaran los fundamentos del uso del giroscopio y acelerómetro, con el respectivo filtro.

- **Giroscopio:**

  Este es un dispositivo que permite medir el ángulo de rotación girado por un determinado mecanismo. Estos son dispositivos diferenciales, lo que quiere decir que no existe una referencia absoluta, si no que se mide ángulos relativos a una referencia arbitraria. Los utilizado se denominan Giroscopios Vibratorios de efector Coriolis (CVG).

El principio de funcionamiento de un CVG es que un objeto vibratorio tiene a vibrar en el mismo plano incluso si este rota. El efecto Coriolis provoca que el objeto vibratorio ejerza una fuerza sobre el soporte, y midiendo esta fuerza podemos determinar la rotación a la que está sometido el giroscopio.

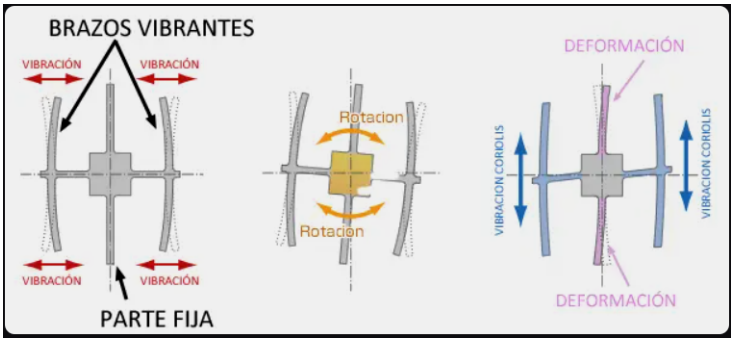


Figure 3: Giroscopio

Una de las consecuencias de emplear la fuerza Coriolis es que los giroscopios vibratorios, a diferencia de otros tipos de giroscopios, no registran el ángulo girado sino la velocidad angular, por lo tanto, para obtener el ángulo de posición del sensor es necesario realizar la integración respecto del tiempo.

Los giroscopios son sensores de respuesta rápida y elevada precisión en tiempos cortos. Además, responden bien a cambios bruscos, y son relativamente inmunes al ruido, siempre en rangos temporales cortos.

- **Acelerómetro:**

  Un acelerómetro es un dispositivo que permite medir la aceleración a la que está sometido. Cualquier cuerpo sometido a una aceleración experimentará una cierta fuerza, empleando este hecho, se entenderá el funcionamiento del mismo.

  Para entender este dispositivo se debe pensar en un cuerpo sólido, en cuyo interior suspendemos una masa sujeta por muelles al cuerpo exterior.

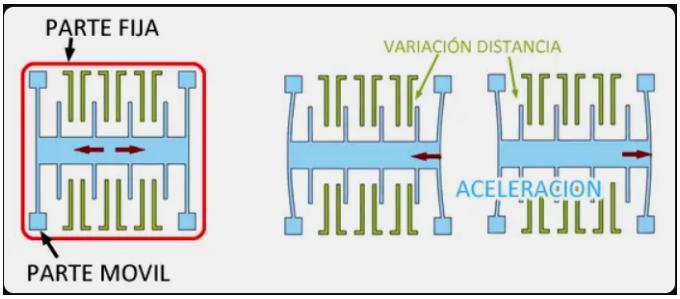


Figure 4: Acelerómetro

Al aplicar una aceleración al conjunto la masa suspendida ejercerá una fuerza sobre los muelles causando que uno se contraiga y otro se elongue, por lo que la posición relativa de la masa dentro del sensor variará. Este desplazamiento de la masa libre interior puede ser medido para determinar la magnitud de la aceleración. El desplazamiento será proporcional a la aceleración soportada, y se mantendrá constante mientras la aceleración sea constante.

El sensor se ve afectado por la gravedad terrestre, que supone una aceleración de aproximadamente $9.81\,\frac{m}{s^2}$ en la superficie de la tierra, que por supuesto es registrada constantemente por el mismo. Dado esto se puede utilizar este efecto para poder determinar la orientación.

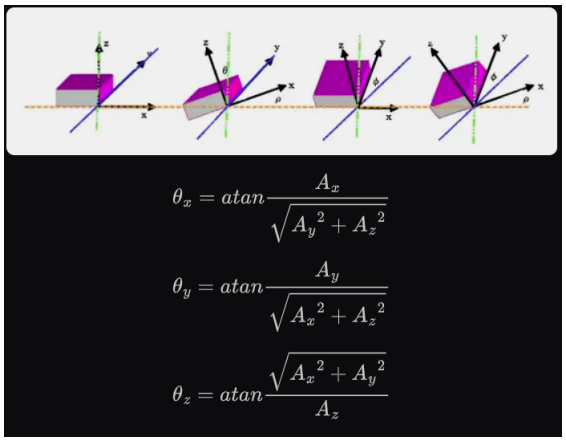


Figure 5: Orientación

Como se menciono anteriormente se hace uso del modulo MPU6050, el cual incorpora un acelerómetro y un giroscopio, conociéndose como IMU (Unidad de Medición Inercial), capaz de poder medir aceleración, velocidad y orientación de un sistema. Al presentar estos dos dispositivos, el modulo tiene 6 grados de libertad (DOF), lo que representa la cantidad de magnitudes independientes que es capaz de medir.

Se debe tener en cuenta que cuando se manejan estos dispositivos se suelen usar los ángulos de Tait-Bryan, en los que la orientación se representa como tres rotaciones ortogonales en torno al eje X (roll), Y (pitch), y Z (yaw).

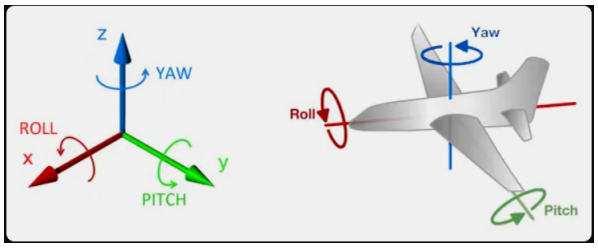


Figure 6: Rotaciones

Para poder tener las ventajas en tiempos cortos del giroscopio y las ventajas a medio y largo plazo del acelerómetro es necesario que se combinen y filtren las señales, siendo uno de los filtros mas utilizado en este tipo de caso el de Kalman.

- Los giroscopios funcionan muy bien para movimientos cortos o bruscos, pero al usar giroscopios de vibración que realmente miden la velocidad angular, y obtienen el ángulo por integración respecto al tiempo, acumulan los errores y el ruido en la medición, por lo que a medio o largo plazo tienen deriva (drift).
- Los acelerómetros no tienen deriva (drift) a medio o largo plazo, ya que realizan la medición medida absoluta del ángulo que forma el sensor con la dirección vertical, marcada por la gravedad. Sin embargo, se ven influenciados por los movimientos del sensor y el ruido por lo que no son fiables a corto plazo.

Ya con toda esta información se pasará a explicar al código implementado para el manejo del modulo.

```
class KalmanFilter {
  public:
    float Q_angle = 0.001f;
    float Q_bias = 0.003f;
    float R_measure = 0.03f;
    float angle = 0.0f;
```

```
7       float bias = 0.0f;
8       float P[2][2] = {{0.0f, 0.0f}, {0.0f, 0.0f}};
9
10      float getAngle(float newAngle, float newRate, float dt) {
11        float rate = newRate - bias;
12        angle += dt * rate;
13
14        P[0][0] += dt * (dt*P[1][1] - P[0][1] - P[1][0] + Q_angle);
15        P[0][1] -= dt * P[1][1];
16        P[1][0] -= dt * P[1][1];
17        P[1][1] += Q_bias * dt;
18
19        float S = P[0][0] + R_measure;
20        float K[2] = {P[0][0] / S, P[1][0] / S};
21        float y = newAngle - angle;
22        angle += K[0] * y;
23        bias += K[1] * y;
24
25        P[0][0] -= K[0] * P[0][0];
26        P[0][1] -= K[0] * P[0][1];
27        P[1][0] -= K[1] * P[0][0];
28        P[1][1] -= K[1] * P[0][1];
29
30        return angle;
31      }
32  };
33
34  KalmanFilter kalmanPitch;
35  KalmanFilter kalmanRoll;
36
37  uint32_t timer_kalman;
```

Se empieza creando una clase, la cual luego sera instanciada para cada eje con el que se trabajara, esta realizara el calculo de los ángulos mediante el filtro de Kalman. Se debe tener en cuenta que el filtro es un estimador probabilístico de estados, por lo tanto, se hará uso de matrices de covarianza (Q y R).

Las matrices "Q_bias" y "Q_angle" representan el ruido interno del giroscopio, siendo la primera la varianza en la deriva del giroscopio, y la segunda la varianza en la predicción del ángulo. Mientras que "R_Measure" representa el ruido de la medición externa, es decir, el acelerómetro.

Luego, "angle" y "bias" seran las variales de estado, siendo la primera la posición real de la mano, y la segunda la cantidad de grados por segundo de error que el giroscopio está generando. Continuamos definiendo la matriz "P[2][2]" que guarda la "incertidumbre" del filtro. En cada ciclo, el filtro actualiza esta matriz para saber qué tan seguro está de sus propios cálculos.

Pasamos a definir la función "getAngle()", esta tomara tres argumentos: la medición del acelerómetro (newAngle), la velocidad del giroscopio (newRate) y el tiempo transcurrido (dt). Funciona en dos fases clásicas: Predicción y Actualización.

Primero se resta el error conocido (bias) a la velocidad del giroscopio (newRate) para obtener una velocidad limpia (rate), luego, se integra la velocidad en el tiempo y se suma al último ángulo conocido, siendo una estimación a priori, para actualizar la matriz de incertidumbre.

Ahora se toma el ángulo medido por el acelerómetro para corregir la predicción. Empieza realizando una estimación del error total (S) y con esto se calcula la ganancia de Kalman (K). Esta ultima matriz decide matemáticamente quien es mas confiable, si el acelerómetro es muy ruidoso, K sera pequeña.

Continua calculando la diferencia matemática entre lo que el acelerómetro midió (newAngle) y lo que el filtro predijo (angle). Se multiplica esa diferencia por la Ganancia de Kalman y se lo suma a la estimaciones. Esto corrige de forma perfecta tanto el ángulo final como la deriva matemática (bias) del sensor. Esta es la estimación a posteriori. Finalmente, como se acaba de usar un dato real (el acelerómetro) para corregir el rumbo, la incertidumbre disminuye. Se actualiza la matriz P restándole la incertidumbre que se acaba de resolver.

```
void inicializarMPU() {
  Wire.begin(21, 22);
  if (!mpu.begin(0x68, &Wire)) {
    Serial.println("Error al iniciar MPU6050.");
    while (1) delay(10);
  }

  mpu.setAccelerometerRange(MPU6050_RANGE_2_G);
  mpu.setGyroRange(MPU6050_RANGE_500_DEG);
  mpu.setFilterBandwidth(MPU6050_BAND_21_HZ);
  timer_kalman = micros();
}
```

Para inicializar el modulo, se hace uso de dos pines en el ESP32, el 21 para los datos (SDA) y el 22 para el reloj (SCL), de esta manera se inicia el bus de comunicación I2C. En caso de fallar la comunicación se bloquea intencionalmente la micro para no realizar cálculos que puedan provocar NaN o posibles divisiones por cero, ademas, enviaría valores basuras a la mano pudiendo romperla.

En las siguientes tres lineas se configuran escalas del giroscopio y acelerómetro, mediante registros internos de hardware del MPU6050, y ademas, se activa un filtro pasa-bajos digital que viene incorpodaro por hardware. Luego, Con la función "micro()" se toma el valor actual del reloj interno de la ESP32 en microsegundos y lo guarda en la variable global, siendo esto usada para los cálculos que necesiten integrar el tiempo.

```
AngulosMano leerAngulosFiltrados() {
  sensors_event_t a, g, temp;
  mpu.getEvent(&a, &g, &temp);

  float dt = (float)(micros() - timer_kalman) / 1000000.0;
  timer_kalman = micros();

  float rawPitch = -(atan2(a.acceleration.x,
                    sqrt(a.acceleration.y * a.acceleration.y +
                    a.acceleration.z * a.acceleration.z)) * 180.0) / PI;

  float rawRoll;

  //Proteccion de la zona prohibida
  if (rawPitch > 80.0 || rawPitch < -80.0) {
      rawRoll = kalmanRoll.angle;
  } else {
      rawRoll = (atan2(a.acceleration.y, a.acceleration.z) * 180.0) / PI;
  }

  float gyroRatePitch = -(g.gyro.y * 180.0 / PI);
  float gyroRateRoll = g.gyro.x * 180.0 / PI;

  float smoothPitch = kalmanPitch.getAngle(rawPitch, gyroRatePitch, dt);
  float smoothRoll = kalmanRoll.getAngle(rawRoll, gyroRateRoll, dt);

  // Mapeo por tramos
  int pitchMapeado;
  if (smoothPitch >= 0) {
    //pitchMapeado = map(smoothPitch, 0, 45, 57, 90);
    pitchMapeado = map(smoothPitch, 0, 45, 57, 25);
  } else {
    //pitchMapeado = map(smoothPitch, -25, 0, 25, 57);
    pitchMapeado = map(smoothPitch, -25, 0, 90, 57);
```

```
35    }
36
37    int rollMapeado = map(smoothRoll, -90, 90, 0, 180);
38
39    pitchMapeado = constrain(pitchMapeado, 25, 90);
40    rollMapeado = constrain(rollMapeado, 0, 180);
41
42    if (abs(pitchMapeado - ultimoPitch) >= UMBRAL_RUIDO) ultimoPitch = pitchMapeado;
43    if (abs(rollMapeado - ultimoRoll) >= UMBRAL_RUIDO) ultimoRoll = rollMapeado;
44
45    return {ultimoPitch, ultimoRoll};
46  }
```

Se finaliza la librería con la función "leerAngulosFiltrados()", la cual sera utilizada en el loop para tomar los valores medidos, aplicar trigonometría, fusionar los datos con Kalman, y finalmente estar listos para poder ser enviados.

La función inicia pidiendo todos los registros del MPU6050 mediante la función "getEvent". Luego, calcula el diferencial de tiempo (dt) calculando en microsegundos cuanto tiempo paso desde la última lectura.

En la variable "rawPitch" se calcula el movimiento de subir y bajar la muñeca (Pitch), para esto se debe realizar el arcotangente de la gravedad que reace sobre el eje X en comparación con la hipotenusa de los ejes Z e Y, para luego ser convertidos a grados sexagesimales ($-90°$ hasta $90°$).

Cuando la mano sube o baja más de 80 grados, la fuerza de gravedad se alinea completamente con el eje de rotación. Si se calculase la rotación (Roll) ahí, el arco tangente fallaría intentando dividir por casi cero. Al detectar esta "zona prohibida" ($\pm 80°$), el código ignora al acelerómetro y congela el último valor seguro de rotación (kalmanRoll.angle) hasta que la mano este en una zona geométricamente estable, por dichas razones se hace uso de un condicional sobre los ángulos en el eje X.

Continua tomando valores del giroscopio ("gyroRatePitch" y "gyroRateRoll"), convirtiéndolas en grados por segundo, ya que se mide en radianes por segundo, para poder hacer uso del filtro de Kalman, dando como argumento el ángulo que se obtuvo del acelerómetro, la velocidad del giroscopio y el tiempo transcurrido. Como resultado se obtiene las variables "smoothPitch" y "smoothRoll" que contienen los ángulos absolutos de la mano.

Ya con los ángulos se realiza un mapeo de los valores obtenidos al de los servomotores, ya que se debe tener en cuenta que en estos motores (MG946R) los valores van desde 0 a 180. Para los movimientos verticales (pitch) el servo solo debe cubrir el rango de 25 hasta 90, siendo 57 la posición media, por lo que, se mapea en dos tramos, de la mitad para arriba y la mitad para abajo, de esta manera se refleja de forma fluida los movimientos medidos en la mano 3D. Para el caso de la rotación (roll), solo se debe mapear todos los valores (-90 a 90) al rango completo del servo (0 a 180). Cabe destacar el uso de la función "constrain" que recorta cualquier valor que se excede del rango.

Para finalizar, se realiza una comparación del nuevo calculo con el último enviado, si la diferencia supera un cierto umbral (configurado en $2°$) se actualiza la nueva información, en caso de no superar el umbral no se actualiza para no estar constantemente enviando pequeñas variaciones de ángulo y hacer zumbar el servomotor, evitando desgaste innecesarios en los engranajes.

3.2 Receptor

3.2.1 Diseño

Antes de pasar a explicar el conexionado y el código, detallaremos el diseño en 3D realizado, para esto se hizo uso de Solid Edge, el cual es un programa de diseño desarrollado por Siemens, para la realización de objetos 3D, asistido por computadora (CAD).

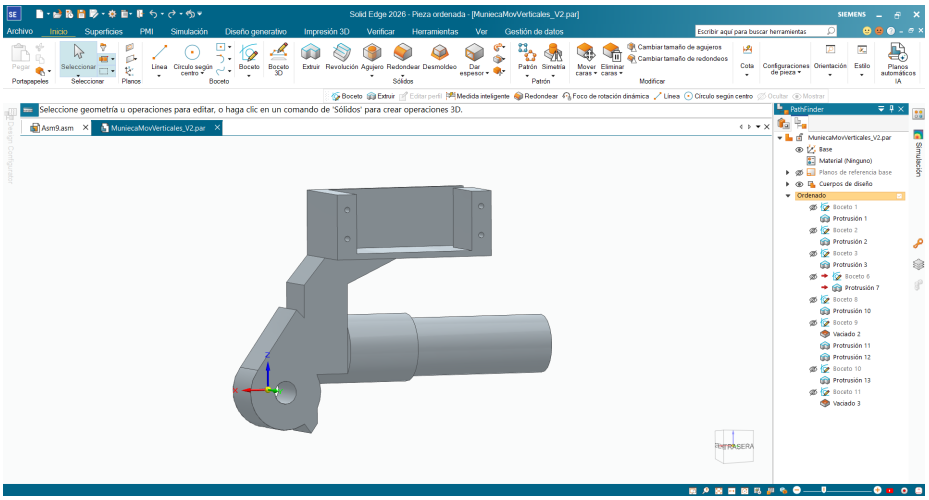
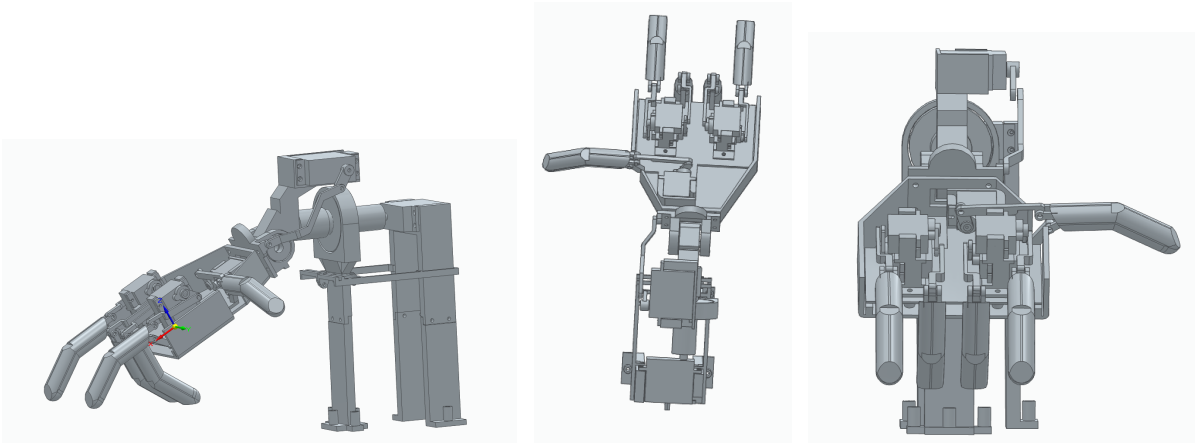


Figure 7: Solid Edge

Este programa resulta muy amigable con el usuario, donde se deben realizar, en un plano, los bocetos del diseño del objeto deseado, para luego mediante la Extrusión, poder extenderlo de las 2 dimensiones a las 3 dimensiones. En un mismo objeto se van formando bocetos en distintas partes del mismo, y mediante la extrusión, se van haciendo vaciados o protrusiones, dandole asi la forma deseada. De esta manera, se desarrollo la siguiente estructura:



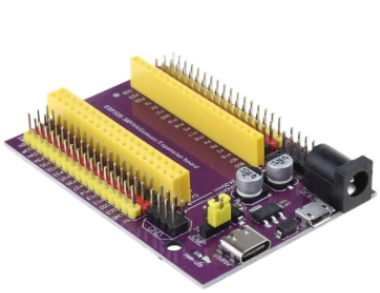
(a) (b) (c)

Figure 8: Estructura de la mano

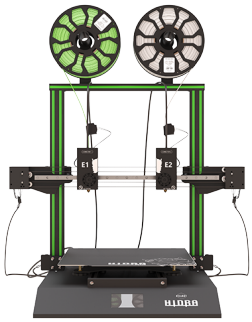
El diseño consta de dos soportes, a unos 17cm de altura, que sostendran toda la estrucutura. Para la rotación se engancha un brazo (Figura 7) con el servo de forma directa, para facilitar el movimiento se hace uso de un ruleman (20cm de diametro interior), luego este se une con la mano mediante un tornillo que pasa entre los dos. Ademas, este contiene el servo que se engancha con la mano, el cual realizara los desplazamientos verticales.

Por ultimo, dentro de la mano, se tiene una base que unira todo el conjunto de servo SG90, los servos se ubicaran de a pares de forma vertical, uno en cada dirección, mediante un brazo se enlaza al dedo, siendo que el dedo esta atornillado a la base. El unico que es distinto, es el dedo pulgar, donde el servo se atornilla directo a la base.

Cabe recalcar que tambien se diseñaron pequeños soportes, que van enganchados a los soportes mas grandes, para poder sostener la base de la micro de forma sencilla. Todos los diseños realizados fueron impresos en la impresora Hellbot Hidra.



(a) Base ESP32



(b) Impresora 3D Hellbot Hidra

Figure 9

3.2.2 Conexionado y Código

Gracias al uso de la Base de la ESP32 (9), resulto mas sencillo el conexionado de los servos y la alimentación del todo el circuito, ya que este presenta pines en la posición necesaria para poder conectar los servos (Señal, VCC y GND) y la placa, puediendose alimentar con dos tipos de USB (MicroUSB y USC-C) y con un miniplug, siendo el ultimo el requerido para poder lograr que todo funcione con normalidad, por su capacidad de entregar entre 6.5 a 16V. Ademas, se debe tener en cuenta que la fuente a utilizar debe poder administrar la potencia necesaria, siendo los valores electricos de funcionamiento los siguientes:

- SG90: entre 5/6 V y 100/250 mA sin carga hasta 500 mA bajo bloqueo.
- MG946R: entre 4.8/6 V y 170 mA sin carga hasta 1.2 A bajo bloqueo.
- ESP32: con 5V y entre 300/500 mA con Wifi activo.

Dado que los SG90 nunca llegaran al bloqueo se los considera con consumo normal, en cambio, se debe tener mas en cuenta la corriente de bloqueo de los MG946R, y mas que nada el de los movimientos verticales, ya que es el que mas peso debe cargar. La fuente debe ser capaz de administrar, considerando todo lo anterior, al menos unos 3A para que todo pueda funcionar con normalidad, hasta en los casos de mucho consumo.

Pasando el código, se empieza incluyendo las librerias necesarias y definiendo los pines/objetos necesarios.

```
#include <esp_now.h>
#include <WiFi.h>
#include <ESP32Servo.h>

const int pinServoPulgar = 25;
const int pinServoIndice = 26;
const int pinServoMedio = 27;
const int pinServoAnular = 12;
const int pinServoMenique = 13;
const int pinServoMuniecaVertical = 32;
const int pinServoMuniecaRotacional = 33;

Servo servoPulgar;
Servo servoIndice;
Servo servoMedio;
Servo servoAnular;
Servo servoMenique;
Servo servoMuniecaVertical;
Servo servoMuniecaRotacional;
```

Primero veamos para que sirve cada librería:

- **Esp Now:** Es la librería oficial de Espressif (fabricante de ESP32), encargada del protocolo de comunicación inalámbrica punto a punto que funciona sin necesidad de conectarse a un router, usando direcciones MAC. Esta permite enviar paquetes de datos muy pequeños a una alta velocidad y con bajísimo consumo de energía.
- **Wifi:** Es la librería estandar de Arduino para manejar el hardware de radio Wi-Fi integrado en la placa ESP32. Para que ESP-NOW funcione, la antena Wi-Fi de la ESP32 debe estar encendida físicamente.
- **ESP32Servo:** Es una librería diseñada exclusivamente para controlar servomotores usando la arquitectura interna de la ESP32. No se usa la librería clasica de Arduino ¡Servo.h¿ ya que ESP32 no tiene comando analógicos simples, usa un periférico interno muy complejo llamado LEDC (LED Control) y temporizadores (Timers) de hardware para generar señales PWM.

Dado los pines del ESP32

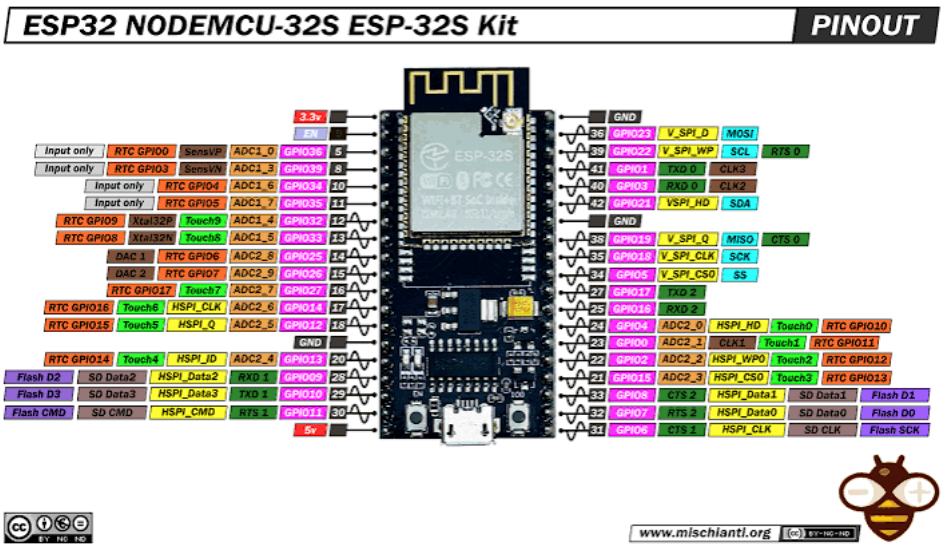


Figure 10: ESP32 Pines

se utilizaron los que se visualizan en el código por comodidad a la hora de realizar la conexión, son los que se encuentran mas cerca a los cables, ya que estos son de unos 30cm, y hay que contemplar que a la hora de que realice los movimientos la mano estos no se desconecten. Ademas, estos pines no cumplen una función especial que pueda interferir con el uso que se le quiere dar.

Luego de la definición de pines, se declaran objetos Servo, los cuales contienen las funciones necesarias para poder realizar las distintas acciónes sobre los servos.

Continuamos con un detalle fundamental, la estructura de los datos

```
typedef struct mensaje_brazo {
  uint16_t valPulgar;
  uint16_t valIndice;
  uint16_t valMedio;
  uint16_t valAnular;
  uint16_t valMenique;
  uint16_t valMuniecaVertical;
  uint16_t valMuniecaRotacional;
} mensaje_brazo;

mensaje_brazo datosRecibidos;

```

```
volatile bool nuevosDatos = false;
bool sincronizado = false;
```

se crea una estructura que empaqueta los datos de las variables que se necesitan, definiendo en cada una el tipo de dato, para luego instanciarlo en una variable. Cabe recalcar que este tipo de estructura tiene que ser identica tanto en el transmisor como receptor, si no se cumple esto habra uan falla en la lectura de datos recibidos. Tambien, se crean dos variables booleanas para poder sincronizar la recepción.

```
float alfaDedos = 0.2;
float alfaMunieca = 0.3;

// Posiciones Actuales vs Objetivo
float actPulgar=25, actIndice=25, actMedio=25, actAnular=25,
actMenique=25, actMuniecaVertical=57, actMuniecaRotacional = 90;

float objPulgar=25, objIndice=25, objMedio=25, objAnular=25,
objMenique=25, objMuniecaVertical=57, objMuniecaRotacional=90;

void OnDataRecv(const uint8_t * mac, const uint8_t *incomingData, int len) {
  memcpy(&datosRecibidos, incomingData, sizeof(datosRecibidos));
  nuevosDatos = true;
}
```

Se crean las variables para poder utilizar el filtro promedio movil exponencial (EMA), el cual se detallara mas adelante. Este filtro se utiliza para poder suavizar los movimientos de los servos, de esta manera se pueden apreciar desplazamientos mas fluidos, por dicha razon se debe tener en cuenta el ángulo objetivo al que se quiere llegar y el actual. Los alfas son factores utilizados para determinar el grado de cambio, mientras mas cercano a uno es, mas rapida sera la respuesta, si es mas cercano a cero, el grado de cambio sera mas lento.

La función "OnDataRecv" es una interrupción que trabaja de forma asincrona, pausa el sistema operativo de la ESP cuando la antena detecta un paquete de radio, para poder ejecutar dichas lineas de código, luego vuelve a la normalidad. En dicho instante, la función "memcpy" vierte directamente en la memoria (datosRecibidos) la fila de bytes que llegaron a la antena.

Sigue la definición del setup(), el cual definirá el hardware de la micro.

```
void setup() {
  Serial.begin(115200);

  ESP32PWM::allocateTimer(0);
  ESP32PWM::allocateTimer(1);
  ESP32PWM::allocateTimer(2);
  ESP32PWM::allocateTimer(3);

  servoPulgar.write(25);
  servoIndice.write(25);
  servoMedio.write(25);
  servoAnular.write(25);
  servoMenique.write(25);
  servoMuniecaVertical.write(57);
  servoMuniecaRotacional.write(90);

  servoPulgar.attach(pinServoPulgar, 500, 2400);
  servoIndice.attach(pinServoIndice, 500, 2400);
  servoMedio.attach(pinServoMedio, 500, 2400);
  servoAnular.attach(pinServoAnular, 500, 2400);
  servoMenique.attach(pinServoMenique, 500, 2400);
  servoMuniecaVertical.attach(pinServoMuniecaVertical, 500, 2400);
  servoMuniecaRotacional.attach(pinServoMuniecaRotacional, 500, 2400);
```

```
24
25    WiFi.mode(WIFI_STA);
26    Serial.print("MAC Address: ");
27    Serial.println(WiFi.macAddress());
28
29    if (esp_now_init() != ESP_OK) {
30      Serial.println("Error inicializando ESP-NOW");
31      return;
32    }
33
34    esp_now_register_recv_cb((esp_now_recv_cb_t)OnDataRecv);
35    Serial.println("Brazo robotico encendido. Esperando transmisor...");
36  }
```

Con la función "allocateTimer" lo que se hace es reservar cuatro temporizadores de hardware interno para el uso exclusivo del PWM, siendo esta señal la que leerán los servos. Se debe saber que los servomotores utilizados se manejan mediante ancho del pulso del PWM, variando su tiempo en ON, estos se posicionaran en el ángulo que se desea.

Gracias al uso de las librerías, mediante la función "write" es que se da la orden de posicionar el servo en cierto ángulo, de esta manera el ESP32 cambiara el ancho de pulso del PWM para posicionarlo. Para lograr esto, tambien se tiene que hacer uso de "attach", el cual asigna el pin físico a cada objeto servo, y ademas, asigna los pulsos eléctricos mínimos y máximos a los ángulos mínimos y máximos, respectivamente.

A partir de aquí se inicializa la comunicación, encendiendo la antena en modo cliente para que escuche los paquetes que llegan, en caso de que falle la inicialización se arroja un error. Finalmente se le dice al sistema que cada vez que escuche un paquete de datos interrumpa el proceso y ejecute "OnDataRecv". Se imprime también la MAC del ESP, ya que el transmisor debe saber a donde deben dirigidos los datos.

Pasando al final del código, veamos el loop(), el cual es la función que se repite constantemente a muy alta velocidad.

```
1  void loop() {
2
3    if (nuevosDatos) {
4      nuevosDatos = false;
5
6      // Mapeo seguro con limites mecanicos de seguridad (Clamping)
7      objPulgar  = constrain(map(datosRecibidos.valPulgar, 0, 4095, 25, 90), 25, 90);
8      objIndice  = constrain(map(datosRecibidos.valIndice, 0, 4095, 25, 90), 25, 90);
9      objMedio   = constrain(map(datosRecibidos.valMedio,  0, 4095, 25, 90), 25, 90);
10     objAnular  = constrain(map(datosRecibidos.valAnular, 0, 4095, 25, 90), 25, 90);
11     objMenique = constrain(map(datosRecibidos.valMenique,0, 4095, 25, 90), 25, 90);
12
13     objMuniecaVertical  = datosRecibidos.valMuniecaVertical;
14     objMuniecaRotacional = datosRecibidos.valMuniecaRotacional;
15
16     // === LOGICA DE SINCRONIZACION ASINCRONA ===
17     if (!sincronizado) {
18       actPulgar = objPulgar;
19       actIndice = objIndice;
20       actMedio = objMedio;
21       actAnular = objAnular;
22       actMenique = objMenique;
23       actMuniecaVertical = objMuniecaVertical;
24       actMuniecaRotacional = objMuniecaRotacional;
25
26       sincronizado = true;
27       Serial.println("Primer paquete recibido. Sincronizacion completa.
```

```
28        Iniciando movimiento.");
29      }
30    }
31
32    // Si se enciende el Brazo antes que el Guante, se queda en reposo aqui
33    if (!sincronizado) {
34      delay(20);
35      return;
36    }
37
38    // === FILTRO EMA MECANICO ===
39    actPulgar  = (objPulgar * alfaDedos)  + (actPulgar * (1.0 - alfaDedos));
40    actIndice  = (objIndice * alfaDedos)  + (actIndice * (1.0 - alfaDedos));
41    actMedio   = (objMedio * alfaDedos)   + (actMedio * (1.0 - alfaDedos));
42    actAnular  = (objAnular * alfaDedos)  + (actAnular * (1.0 - alfaDedos));
43    actMenique = (objMenique * alfaDedos) + (actMenique * (1.0 - alfaDedos));
44
45    actMuniecaVertical  = (objMuniecaVertical * alfaMunieca)  +
46    (actMuniecaVertical * (1.0 - alfaMunieca));
47
48    actMuniecaRotacional  = (objMuniecaRotacional * alfaMunieca)  +
49    (actMuniecaRotacional * (1.0 - alfaMunieca));
50
51    // === EJECUCION ===
52    servoPulgar.write(actPulgar);
53    servoIndice.write(actIndice);
54    servoMedio.write(actMedio);
55    servoAnular.write(actAnular);
56    servoMenique.write(actMenique);
57    servoMuniecaVertical.write(actMuniecaVertical);
58    servoMuniecaRotacional.write(actMuniecaRotacional);
59
60    // Retardo de 20ms para coincidir exactamente
61    con la frecuencia de 50Hz de los Servomotores
62    delay(20);
63  }
```

Lo primero que se hace es revisar la recepción de nuevos datos, en caso de no haberlos se ignora parte del código. Luego, en las primeras 10 lineas se mapean los valores de los dedos leídos del transmisor, a los valores de ángulo que están dentro del rango de movimiento de la mano. En el caso de la muñeca (Vertical y Rotacional) los valores son directos, no hay que mapearlos. Se debe destacar el uso de la función "constrain", el cual sirve para el caso de que, por una interferencia eléctrica, se lea un valor absurdo para el angulo del dedo (120° por ejemplos) excediendo los limites asignados, siendo dicha función la que recortara el valor a los limites asignados (25° a 90°).

En las siguientes 20 lineas se realiza una sincronización de datos, mas que nada al iniciar todo el sistema, receptor y transmisor, para que el brazo no realice movimientos exigentes por leer cualquier tipo de valor, sea porque el transmisor todavía se esta inicializando o directamente esta apagado.

Una vez lograda la sincronización, y la obtención de nuevos datos, se pasa al uso del Filtro EMA, el cual se utiliza para darle fluidez a los movimientos de la mano. La idea del uso de este filtro es que la taza de cambio del ángulo sea mas gradual, por ejemplo, el dedo se encuentra en 25° y debe ir a 90°, considerando un valor de "alfaDedos" de 0.15, el primer instante se moverá un 15% pasando a 34°, luego otro 15% , y así sucesivamente hasta alcanzar el objetivo.

Cada vez que se calcula un nuevo valor de ángulo, con la función "write" se asigna físicamente el ancho del pulso necesario para lograr dicha posición en el servo.

A lo último se puede observar un delay de 20ms, este es utilizado para sincronizar matemáticamente el código con la electrónica analógica del motor, ya que estos suelen trabajar en 50 Hz, lo que equivale a 20ms, de

esta manera se puede garantizar menos vibraciones y máximo torque.

4 Pruebas

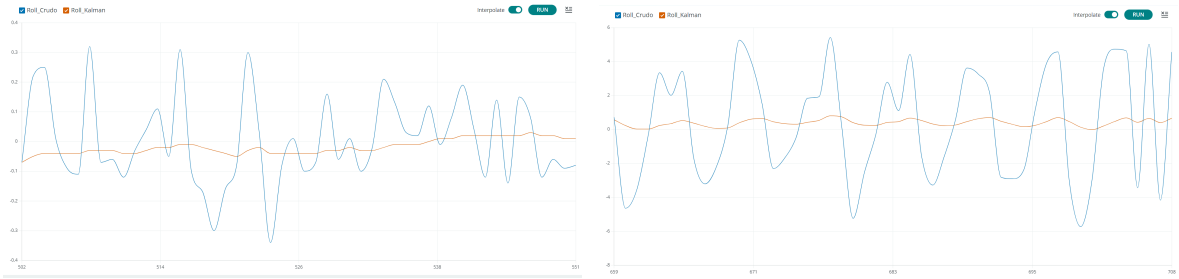
Se observara que tan útil es el uso de los distintos filtros para cada medición realizada en el ESP, primero empezamos con el modulo MPU6050, se observaran las mediciones obtenidas por el micro cuando es modulo esta totalmente quieto y luego con un pequeño movimiento, para poder observar cual es la respuesta al respecto. Para el caso del Pitch resulta:



(a) Pitch Sin Movimiento (b) Pitch Con Movimiento

Figure 11

y para el caso del Roll se observa:



(a) Roll Sin Movimiento (b) Roll Con Movimiento

Figure 12

Ahora se analizará como se comportan los potenciómetros, viendo la respuesta con los datos leídos en crudo y los datos filtrado, lo que da como resultado:



(a) Cambio Lento (b) Cambio Brusco

Figure 13: Respuesta potenciómetro

Se puede ver que con el filtrado la respuesta de la lectura es mas lenta, y es lo que se necesita para el mecanismo de la mano, ya que respuestas rápidas podrían generar roturas en el mecanismo armado o podría dañar los servos, por dicha razón se debe filtrar para evitar movimientos excesivos en la mano.

A continuación se observa la respuesta del sistema completo, tanto el transmisor como el receptor, se apreciaran la forma en que responde cada uno, el tiempo de respuesta, entre que se transmite la información hasta que llega y se replica, entre otros detalles que se puedan apreciar.



(a) Desplazamiento Vertical

(b) Rotación

(c) Subir Dedo

(d) Bajar Dedo

Figure 14: Respuesta sistema

Como se observa en las imágenes, hay un pequeño delay entre que se toma los valores del potenciómetro, se analizan, se transmiten y se replican. Este sera el tiempo de respuesta del sistema, que es de aproximadamente unos 20ms.

5 Conclusiones